



Dynamic configuration Platforms

for digital noise filtering of signals

BACHELOR'S THESIS

submitted in partial fulfillment of the requirements for the degree of

Bachelor of Science

in

Computer Engineering

by

Stefan Dangl

Registration Number 01631845

to the Faculty of Informatics
at the TU Wien

Advisor: Dipl.-Ing. Haris Isakovic

Vienna, 9th April, 2020

Stefan Dangl

Haris Isakovic

Erklärung zur Verfassung der Arbeit

Stefan Dangl

Hiermit erkläre ich, dass ich diese Arbeit selbständig verfasst habe, dass ich die verwendeten Quellen und Hilfsmittel vollständig angegeben habe und dass ich die Stellen der Arbeit – einschließlich Tabellen, Karten und Abbildungen –, die anderen Werken oder dem Internet im Wortlaut oder dem Sinn nach entnommen sind, auf jeden Fall unter Angabe der Quelle als Entlehnung kenntlich gemacht habe.

Wien, 09. April 2020

Stefan Dangl

1 Abstract

This bachelor thesis is about the digital noise reduction of signals. The frequency spectrum of the input signal is analyzed by a micro controller (MCU). The MCU considers in which frequency spectrum the signal is located and in which spectrum noise is located and decides which parts of the spectrum have to be filtered. subsequently a filtering device is configured according to the calculation of the MCU. The filtering device reads the noisy input signal and outputs a corresponding signal with reduced noise.

We designed two implementations of the filtering device. One is running on a FPGA and the other one on a DSP. At the FPGA solution we designed every detail by our one. The DSP solution uses a lot of predefined functionalities. This work analyses the advantages and disadvantages of both implementations.

Our studies show that the implementations both behave as desired with regard to their filter properties. The biggest difference is that the FPGA implementation was a lot of work, but is more flexible. The DSP implementation, on the other hand, was completed in a significantly shorter time, but is not as flexible in its expandability.

The reconfiguration process of the filters runs automatically. Therefore a Host Computer (Server) is used as an linking device between the MCU and the filtering devices. Changes the frequency of the Input signal, the new frequency is recognised by the MCU. The MCU sends his considered cut off frequencies to the Server. The Server does some calculation and reconfigures the filter instruments.

This work is not only theoretically, but also contains real code, visible at Git Lab [17]

Contents

1	Abstract	2
2	Introduction	4
2.1	Motivation	4
2.2	Proposed Solutions	4
2.3	Overview of the Thesis	4
3	Related Work	5
3.1	Dynamic Analogue noise reduction Circuits	5
3.2	Dynamic Digital noise reduction Algorithms	5
3.3	Other Aspects	5
4	Background	6
4.1	Altera DE2-70 FPGA-Board	6
4.2	Wondom DSP Board	6
4.3	Frequency Response of the Filter Instruments	7
4.4	Microcontroller: PJRC - Teensy 3.6	9
4.5	Server	9
5	Methodology	10
5.1	Overview	10
5.2	Noise reduction Algorithm	10
5.3	Calculation of the Filter Coefficients	12
5.4	Weak Points of the Algorithm	13
5.5	Reduction of the Sensitivity	14
6	Implementation	15
6.1	Microcontroller-Implementation	15
6.2	Server-Implementation	15
6.3	FPGA-Structure	16
6.4	FPGA/Server-Communication	18
6.5	FPGA-Filter Module	19
6.6	DSP-Implementation	19
7	Result	20
7.1	Noise Reduction - Measurement Setup	20
7.2	Noise Reduction - 100 Hz	20
7.3	Noise Reduction - 1 kHz	21
7.4	Noise Reduction - 10 kHz	22
7.5	Noise Reduction - other Waveforms	23
7.6	Reconfiguration Timing and Latency	24
7.7	Effort and Expandability	25
7.8	Real Use Case	25
8	Conclusion	26

2 Introduction

2.1 Motivation

There are many reasons why signals, i.e. from a sensor or a music instrument can be overlaid with noise. The detailed description of those noise sources are not part of this work and can be read in books like *Rauschen* from Rudolf Müller [15] or *Electronic Noise and Interfering Signals* from Gabriel Vasilescu [16]. There are also many other good books.

In the case of a sensor, noise can lead to not exactly recognised data. That could further lead to, e.g. inaccurate controlling. In the case of a music instrument, noise leads to a recognisable deterioration in audio quality. In both cases the filtering of the signal for the purpose of noise reduction is useful.

2.2 Proposal Solutions

A simple approach would be the use of static filters. They can be realised with analogue components like resistors, capacitor and operation amplifiers or with digital Algorithms. A disadvantage of such static filters is that the filter curve does not adapt to the input signal. That means that always the same frequency components are weakened and amplified regardless of the input signal.

Dynamic Filters are used if the filter curve should adapt to the input signal. Again there are a lot of possibilities to implement such dynamic filters. E.g. digital filter algorithms can be implemented using DSPs (Digital Signal Processors) or FPGAs (Field Programmable Gate Arrays).

It's very simple to implement static signal processing using DSPs but DSPs are quite limited on that area. FPGAs can be used for a lot of use cases. They can be used for simple calculations but also as complete CPUs. The disadvantages of FPGAs is that they are more complex to design than DSPs. Because the power of computer systems increased in the past years it's also possible to use only a strong MCU or a PC for the whole task.

The simple noise detection algorithm which is used in this work estimates which regions of the frequency spectrum of the current input signal is noise and which is the actual signal. This estimation leads to a band pass filter which attenuates the noise. The band pass filter consist of a low cut and a high cut. The cut off frequency of the low cut is followed called *lower cut off frequency* and the cut off frequency of the high cut is followed called *higher cut off frequency*.

Alternatives to this algorithm are e.g. analogue dynamic filter circuits. Such circuits are also available as already build ICs. There are also more complex algorithms that provide a better result in noise reduction, but also require more detailed knowledge of the noise spectrum of the input signal. Such Algorithms are e.g. the *Gauß Algorithm* or the *Active Noise Control Algorithm*. Examples of both of them are listed in the next section *Related Work*.

2.3 Overview of the Thesis

In the next section different related works are listed. They are divided in three categories. Further in the section *Background* the used devices and tools are described in detail. The section *Methodology* describes the used noise reduction algorithm in detail and the rough interplay of the devices. In *Implementation* the details of the designed systems are described. At the end of this work the section *Result* shows that the systems work as specified. In addition both implementations, FPGA and DSP are compared in detail.

3 Related Work

3.1 Dynamic Analogue noise reduction Circuits

[1], [2] and [3] also try to filter the noise of signals using dynamic band passes. But instead of digital algorithm they use analogue circuits. The behaviour of that systems are similar to ours anyway. The frequency of the input signal has to be in the range of 0 to 20 kHz to guarantee correct behaviour. Like our system, the circuits are primarily designed for the filtering of audio signals.

[4] is also an analogue circuit. Unlike the previous ones the circuit is designed for the dynamic noise reduction of image luminance signals. That works in a slightly different way than the noise reduction of audio signals.

3.2 Dynamic Digital noise reduction Algorithms

The approach of [5] is quite similar to ours. The difference is that they use just one device instead of three. That means the whole process (analysing, computing and filtering) is integrated into one computational platform. The authors justify that decision by the reason that those platforms become more powerful over the past years. Often they also feature DSP cores.

Instead of the noise reduction filtering of audio signal [6] is about the noise reduction of muscle signals. The authors compare dynamic Gaussian filtering with static band pass filtering. Because the noise filtering of EMG noise is quite similar to the filtering of audio noise, that approach is also quite interesting for our work.

[7] compares three different filter types. Dynamic filters but also static filters. The field of use of that work are also medical health applications such as electrocardiogram (ECG).

[8] offers another approach. The paper uses Active Noise Control (ANC). That means that noise gets analysed to generate a so called anti-noise. The anti-noise overlays the input signal and removes the noise of it (phase cancellation). The counterpoint of this approach is that you need detailed knowledge of the noise. That's one reason why this approach is not applied in our work.

3.3 Other Aspects

[9] introduces two algorithms for dynamic filtering of timevarying sparse signals. The operation area of this work are natural video sequences.

Our work is concentrated on the filtering of the input signal. But an important aspect of digital noise reduction is also the noise generated by the used devices, specially the Analog-Digital (ADC) and Digital-Analog Converters (DAC). [10] and [11] bring out the contributing factors to the noise and suggests techniques to minimize the noise in an ADC circuit.

4 Background

4.1 Altera DE2-70 FPGA-Board

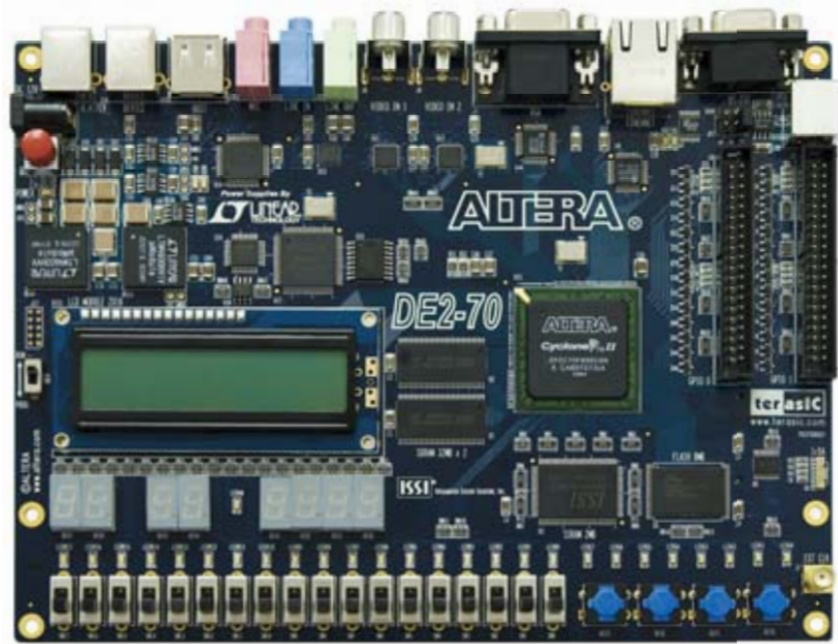


Fig. 1: FPGA-Board: Altera DE2-70 [12]

The Altera DE2-70 FPGA Board is a development board. It contains the Cyclone II FPGA, a RS232 Interface and the WM8731 relevant to us but also many other features like usb modules. The Cyclon II FPGA was build with 90-nm technology. The maximum clock frequency usable is 50 MHz but to reduce power consumption our implementation runs only at 12 MHz.

The WM8731 Audio chip reads the input signal and generates the output signal. It offers a variety of configuration options. In order to obtain the best possible signal resolution, a resolution of 24 bit and a sampling rate of 96 kHz was chosen. This resolution corresponds to the standard for high-resolution audio signals. For comparison, the widespread Audio-CD standard only offers a resolution of 16 bit and 44.1 kHz.

The Cyclone II FPGA was designed using the chargeable program *Quartus II*, published by Altera (Intel). For simulation, the HDL simulation software *Modelsim* was used. All of the code for the Cyclone II was written in VHDL.

4.2 Wondom DSP Board

The most important part of the Wondom DSP board is an ADAU1701 DSP chip from Analog Devices. The board offers 2 ADCs and 4 DACs, all of them operate with a maximum sampling frequency of 96 kHz and a maximum resolution of 24 bit. Here, too, we opted for the highest possible temporal and value-specific resolution in order to obtain the best possible signal quality and to keep the quantization noise as low as possible.

The Wondom DSP board was programmed using the free software *Sigma Studio*. *Sigma Studio* offers a well usable graphical programming interface, but also enables the writing of scripts in the programming languages C# and Visual Basic. Those script can be used to configure the

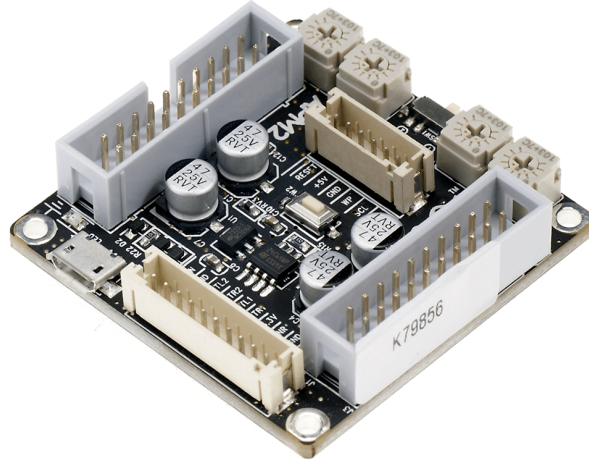


Fig. 2: Wondom DSP Board [13]

DSP board, but can also be used to automate some processes, like we did. Unfortunately *Sigma Studio* is not available for Unix systems.

4.3 Frequency Response of the Filter Instruments

Figure 3 and 4 show the measured filter curves of the FPGA board and the DSP board. It can be seen that both systems have a very sharp drop in amplitude in the range around 24 kHz. More specifically, frequencies from 22 kHz are cut off in the case of the DSP board and frequencies from 27 kHz are cut off in the case of the FPGA board. This behavior has no negative impact on the quality of the output signal. However, it should be noted that, as shown in the section *result*, the filtering of the high frequencies of the DSP board has a slightly positive impact of the quality of the output signal.

As described, both filter instruments sample the input signal with 96 kHz. This means that at a frequency of 24 kHz there are only 4 sampling points available per period. Although these are sufficient to recognize the frequency, the quantization noise is correspondingly high. Therefore, it is better to ignore these frequency components.

Furthermore, a high pass characteristic can be seen in both cases. The cut-off frequency is about 10 Hz in the case of the FPGA board and about 20 Hz in the case of the DSP board. In addition, the amplitude falls with decreasing frequency in the case of the FPGA board with 20 dB/dec and in the case of the DSP board Boards with 120 dB/dec. The measured phase profiles behave analogously.

It should be noted that primarily the AD and DA converter, but also the measurement setup, i.e. the cables used for the measurements, are responsible for the band-pass behavior.

From an audio-technical point of view, the low-pass behavior of the systems does not affect the quality of the systems at all, since the frequency spectrum that people can hear only extends from 20 Hz to 20 kHz. The high-pass behavior would be unnoticed in most cases, but it reduces the frequency parts called sub-bass.

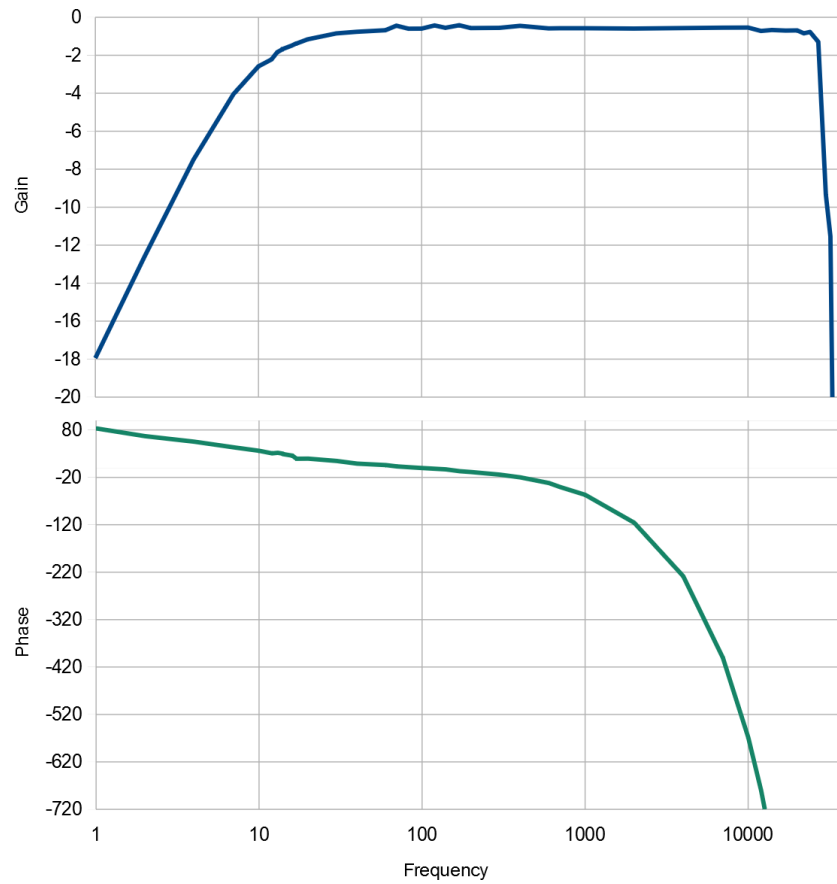


Fig. 3: bode diagram of the FPGA-Board

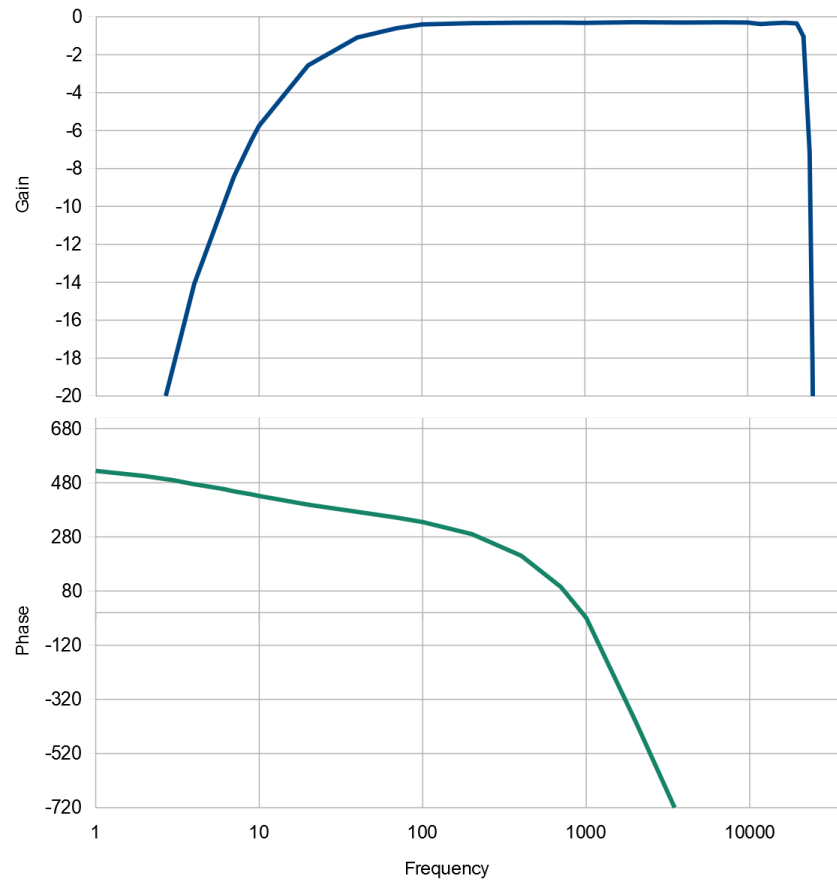


Fig. 4: bode diagram of the DSP-Board

4.4 Microcontroller: PJRC - Teensy 3.6

The microcontroller *Teensy 3.6* from PJRC uses FFT to measure the frequency spectrum of the input signal. The MCU contains a 32 bit, 180 MHz ARM Cortex-M4 processor.



Fig. 5: Microcontroller Teensy 3.6 [14]

In order to obtain a resolution that is sufficiently good for analysis purposes, the *Audio Adapter Board for Teensy 3.0 - 3.6* with a resolution of 44.1 kHz and 16 bit is used. This corresponds, for example, to the resolution of the Audio-CD standard. The audio adapter communicates with the MCU via I2C and contains a slot for SD cards, which is used to buffer the audio data.

According to Shannon's Law, sine waves can be reconstructed if their frequency is less than half the sampling frequency. Accordingly, sinusoidal oscillations up to 22.05 kHz can be detected, which satisfies the requirements.

The code for the Teensy can easily be written using the Arduino IDE, with some additional libraries provided by PJRC, especially in the audio area. For example, a FFT function is offered, which is used in this work.

The FFT function has a frequency resolution of 43 Hz and works in a frequency spectrum from 0 to 21973 Hz. It should be noted that the frequencies are at a constant distance and not logarithmically distributed, as is usual in such applications. This can be seen in the example of the noise deduction algorithm in the next section.

4.5 Server

A standard Windows 10 PC with Intel i5 processor (8th generation) and 8 GB RAM serves as the server. On the server run the programs *Quartus II* to configure the FPGA board, *GNU Octave* to calculate the filter coefficients for the FPGA board and *Sigma Studio* to (re)configure the DSP board. The automatic reconfiguration is managed by a bash file. The main reason why we chose a Windows PC instead of an Unix like system is that unfortunately Sigma Studio is not available for Unix systems.

5 Methodology

5.1 Overview

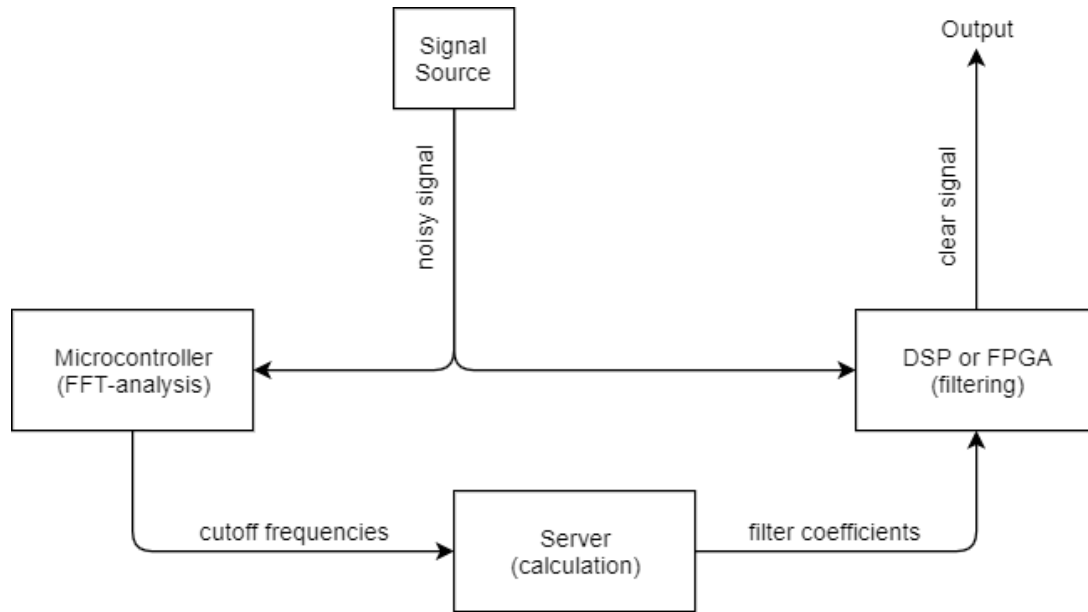


Fig. 6: Structural diagram of the implemented system

As can be seen in Figure 6, the system described in this work consists of three components. The noisy input signal is connected to both the MCU and the filter instrument. In this work a DSP and a FPGA are used as independent filter instruments. At the end of the work, the two approaches are compared in detail.

The MCU analyzes the noisy input signal using a FFT algorithm and estimates the frequency range in which the signal is located. A band restriction is then calculated for that frequency range. The MCU therefore calculates the upper and lower cut-off frequency of a band pass. The algorithm is described in detail in the subsection *Noise reduction Algorithm*. The cut-off frequencies calculated by the MCU are transmitted to the server.

For the FPGA implementation, the server uses the math program *GNU Octave*. *GNU Octave* is a free alternative to *MATLAB*. In our case it is used to calculate suitable filter coefficients for the cut-off frequencies obtained. The server sends the calculated filter coefficients to configure the filter instrument. In case of the DSP implementation *Sigma Studio* is used again to automatically reconfigure the DSP.

The appropriately configured filter instrument reads the noisy input signal, filters it and, in the ideal case, outputs a signal with a lower noise component. Certain frame conditions are required for the input signal. These will be described in more detail later.

5.2 Noise reduction Algorithm

The following figures show operations on the frequency spectrum of the input signal, done by the MCU. It should be noted that the frequency is plotted logarithmically in the range between 20 Hz to 20 kHz on the x-axis. The y-axis represents the amplitude of the spectral lines.

Figure 7: After performing the FFT, a frequency spectrum is obtained. In this example the input signal is a 1 kHz sine wave, which is overlaid with a 3 kHz sine wave and noise.

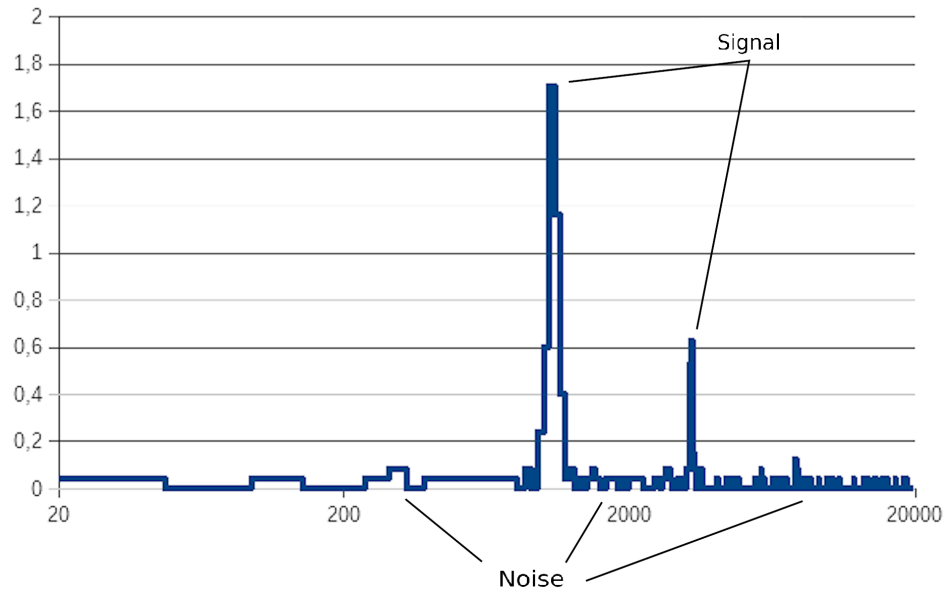


Fig. 7: Spectrum of the input signal

Figure 8: To calculate the mean value of the spectrum, the individual spectral values are added and then divided by the number of measurements unequal to 0 (arithmetic averaging).

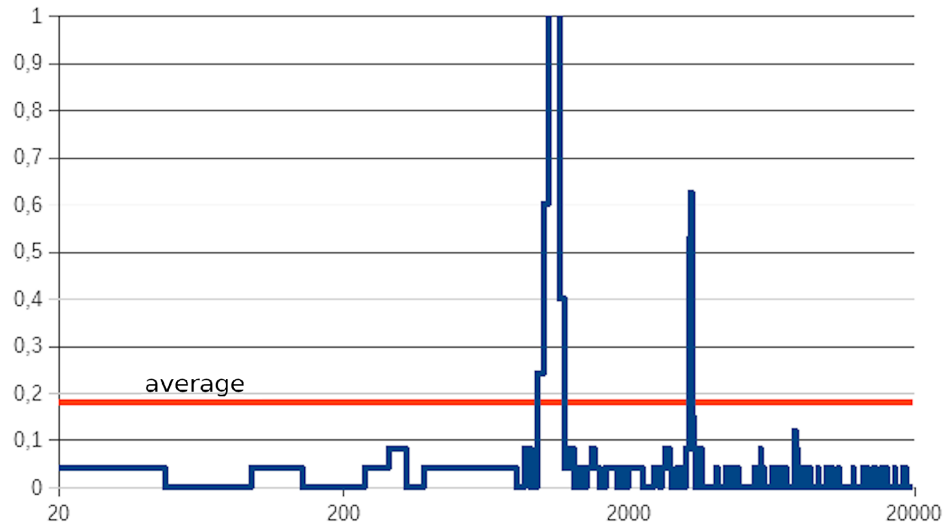


Fig. 8: Calculation of the mean value

Figure 9: The lower cut-off frequency of the band pass is calculated by iterating through the individual spectral values. The iteration starts at the lowest frequency (0 Hz). If the value of a spectral line is greater than or equal to the previously formed mean value, it is assumed that this frequency already belongs to the signal. This chosen frequency is used for the further calculation. It is guaranteed that such a value is found by iterating through the frequencies.

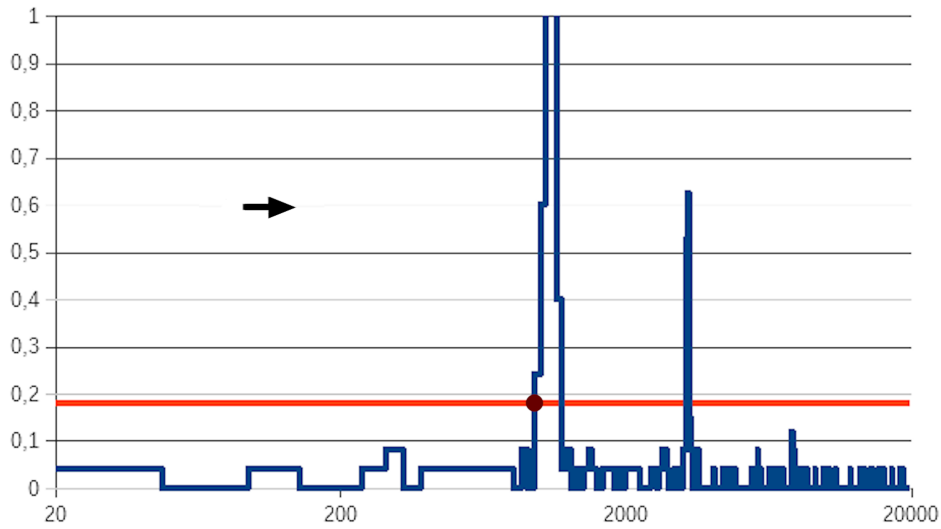


Fig. 9: Calculation of lower cut-off frequency

Figure 10: Just as the lower cutoff frequency was calculated, the upper one is also calculated. However, iteration starts at the highest frequency. The frequency is gradually reduced until a spectral value is found that is greater than or equal to the mean value.

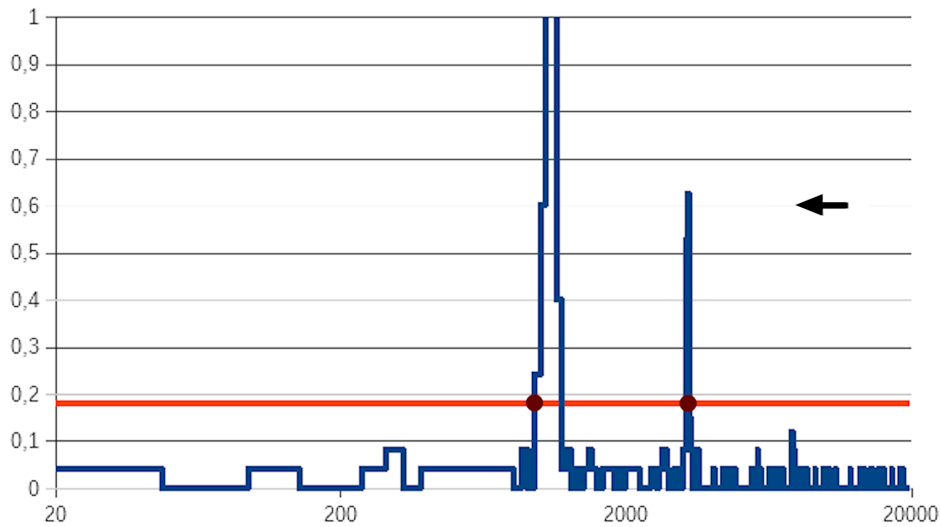


Fig. 10: Calculation of higher cut-off frequency

Figure 11: The filtering should influence the signal as little as possible. Accordingly, the determined value of the lower cut-off frequency is halved and that of the higher cut-off frequency is doubled before the values are sent to the server.

5.3 Calculation of the Filter Coefficients

After the server has received the cut-off frequencies, the server executes a simple MATLAB script. An analog first order Butterworth band pass is generated. The continuous transfer function

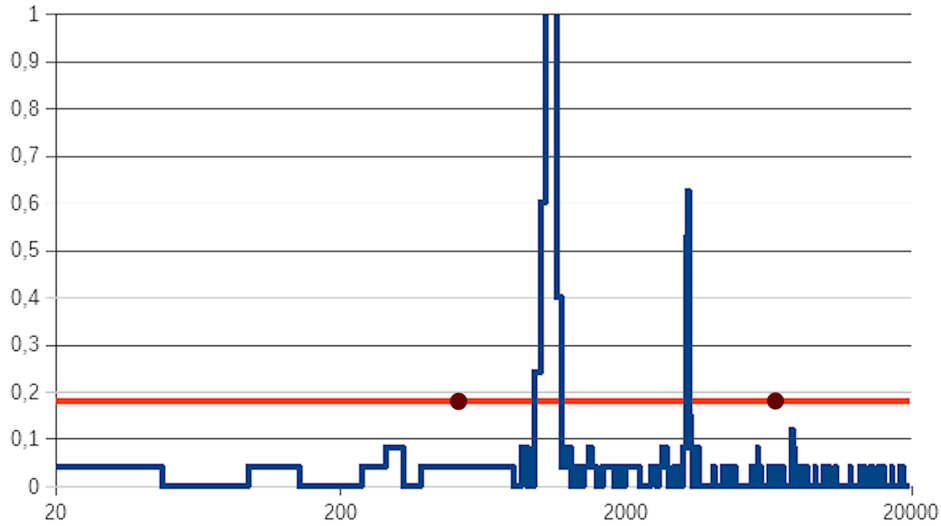


Fig. 11: Adjusting the cut-off frequencies

obtained is then converted to a discrete transfer function, using the 96 kHz sampling frequency. Further the discrete transfer function is converted into a state space representation. The elements of the obtained matrices are the coefficients of the filter. These coefficients are used to configure the filter instruments. This leads to a filtered output signal shown in figure 12.

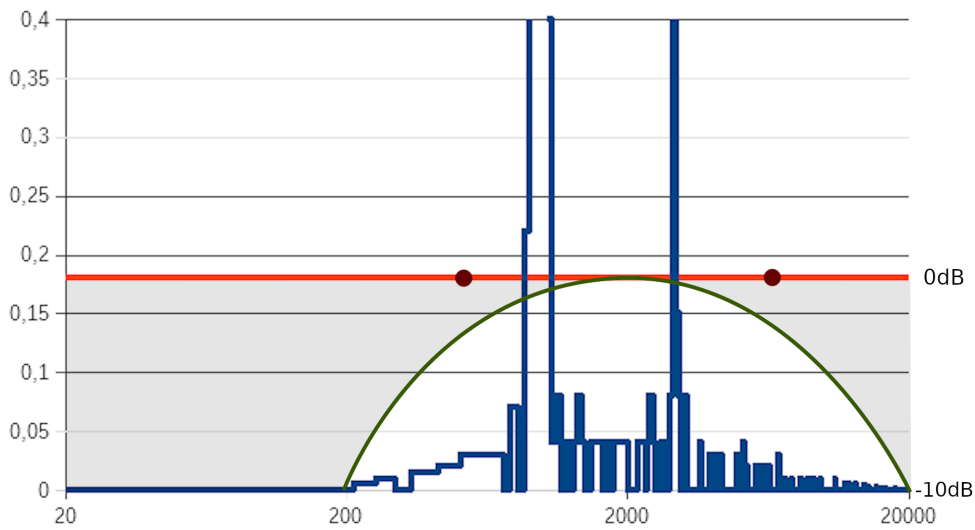


Fig. 12: filtered spectrum

Instead of a first-order bandpass, higher-order ones can also be used. The noise component can be filtered out even better. Higher order filters are not discussed in this work. However, it is possible to implement them with very little effort.

5.4 Weak Points of the Algorithm

For the sake of completeness, two situations are listed here in which the algorithm does not work correctly. In these cases, the use of the algorithm is not recommended.

If the noise component is too strong, then the spectral values of the noise in a certain frequency range are stronger than those of the signal. Then it could happen that the noise is incorrectly recognized as the signal. Though this problem presupposes a very bad signal-to-noise ratio.

Another weak point is the usage for signals that cover the entire frequency spectrum from 20 Hz to 20 kHz. Assuming no noise is superimposed on this signal. Then, as described, the algorithm would find the lowest and the highest frequency at which the mean value is reached the first time. As a result, a band pass is calculated as intended. This resulting filter does not filter noise, but rather important frequency ranges of the signal. In the case of an audio signal, it would remove important bass tones and highs. From a musical point of view, the resulting effect can be called as telephone effect.

5.5 Reduction of the Sensitivity

As is shown in the section *Result*, the algorithm is configured in such a way that it enables a relatively large number of harmonics and, accordingly, a lot of noise in the case of signals that are not band-limited, for example square waves. The upper cut-off frequency is in that case set relatively high in relation to the fundamental oscillation.

To place the pass band closer to the fundamental oscillation the sensitivity has to be decreased. The following formula can be used for this:

$$avg = \frac{s * value_{max} + avg_{true}}{s + 1} \quad (1)$$

Here avg denotes the comparison value used for the algorithm, $value_{max}$ the maximum spectral value, usually the spectral value of the fundamental oscillation, and avg_{true} the actual mean value of all spectral values. s denotes the reversed sensitivity value. So the higher s is, the more insensitive the algorithm behaves and the closer the cutoff frequencies of the band pass are to each other. In the case of later implementation verification, the following was set:

$$s = 0 \quad (2)$$

This leads to:

$$avg = avg_{true} \quad (3)$$

That means that the actual mean value of the spectral lines serves as the comparison value.

6 Implementation

This section describes the automation chain in detail.

6.1 Microcontroller-Implementation

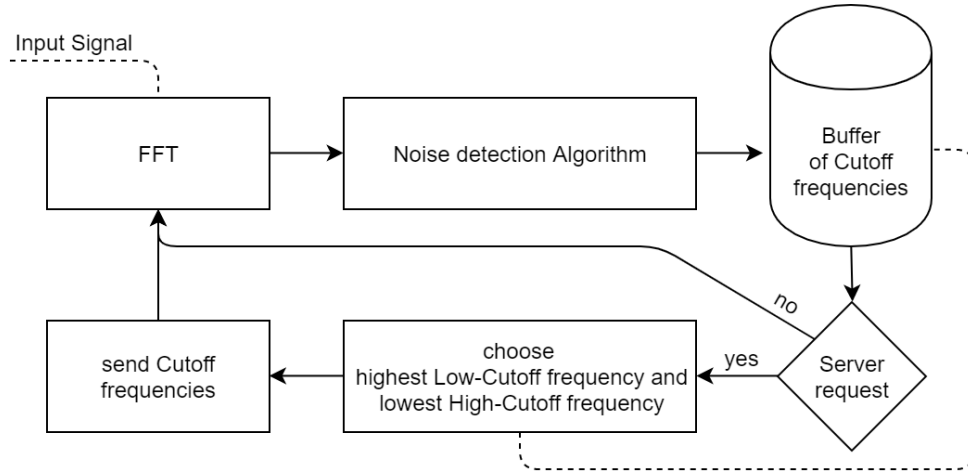


Fig. 13: Implementation of the Microcontroller

The server programs the MCU via serial interface. The calculated cut off frequencies are transmitted to the server via the same. The UART protocol with a baud rate of 9600 is used. If the server needs the data, it sends a request to the MCU. The MCU calculates in an endless loop, with the algorithm described in section *Methodology* the cut-off frequencies for the current input signal.

The newest 20 cut-off frequencies calculated are buffered by the MCU. When the server requests, the MCU iterates through the buffered cut-off frequencies. For the lower cut-off frequency, the maximum value of the buffered lower cut-off frequencies is sent. The minimum value of the buffered upper cut-off frequencies is sent for the upper cut-off frequency. This is done to prevent an alternating change of values which would lead to unnecessary reconfigurations of the filter instruments.

6.2 Server-Implementation

The server was implemented in such a way that a bash file first sends a request to the MCU, as described in the previous subsection. The MCU answers with the filter cut-off frequencies. The server checks whether the received data corresponds to the previously used data. If this is the case, no reconfiguration of the filter instruments is necessary and a new request is sent to the MCU. The MCU sends its current calculated cut-off frequencies again. That process corresponds to a busy waiting structure, which is usually tried to avoid. Since it doesn't matter much, except a slightly higher power consumption, a complex bypass of this structure is not necessary.

If the cut-off frequencies received by the server differ from those previously used for the calculation, the procedure is different for the FPGA and the DSP. In the case of the FPGA, GNU Octave is used to calculate the filter coefficients. These are saved as hexadecimal values in a text document. To send them in the correct order a conversion from Big Endian to Little Endian is done for the 16-digit hex coefficients. In other words the coefficients are mirrored, but saved in the correct order.

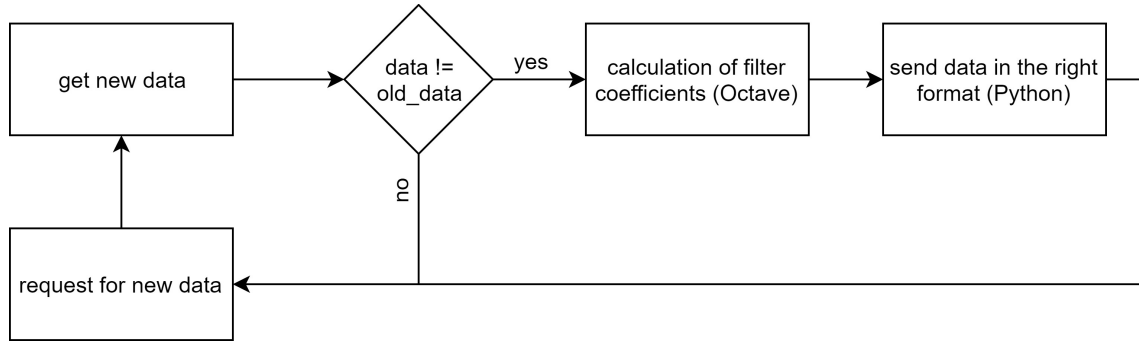


Fig. 14: Server-Implementation (FPGA Version)

Then a small Python script is executed. The Python script reads the file in which GNU Octave wrote the coefficients and sends the individual bytes to the FPGA board via a serial interface. Then the process starts all over again.

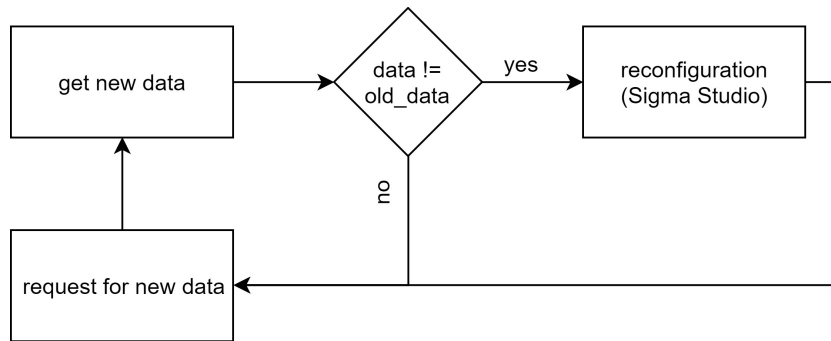


Fig. 15: Server-Implementation (DSP Version)

In the case of the DSP, Sigma Studio is used instead of GNU Octave, which is also used to initialize the DSP. A Sigma Studio Script, written in C#, reads the received cut-off frequencies and automatically calculates the new configuration of the DSP. Sigma Studio also handles the communication with the DSP-board.

6.3 FPGA-Structure

Several modules were implemented in the FPGA. Some of them were developed by us, others were created by third parties (i2c Master, UART-Module) or with the help of the IP Core Wizard built into Quartus II (all floating point modules, the uart-buffers).

An external oscillator located on the FPGA board with a clock frequency of 50 MHz serves as the clock source. This clock is regulated down to 12 MHz by the PLL. This 12 MHz clock powers the entire FPGA design and the external audio chip WM8731.

At initialisation the i2c master is used to configure the audio chip via the i2c bus. During the configuration, it is defined, among other things, that the audio chip should work with a resolution of 96 kHz and 24 bit. For further communication, the FPGA serves as the master and the audio chip as a slave.

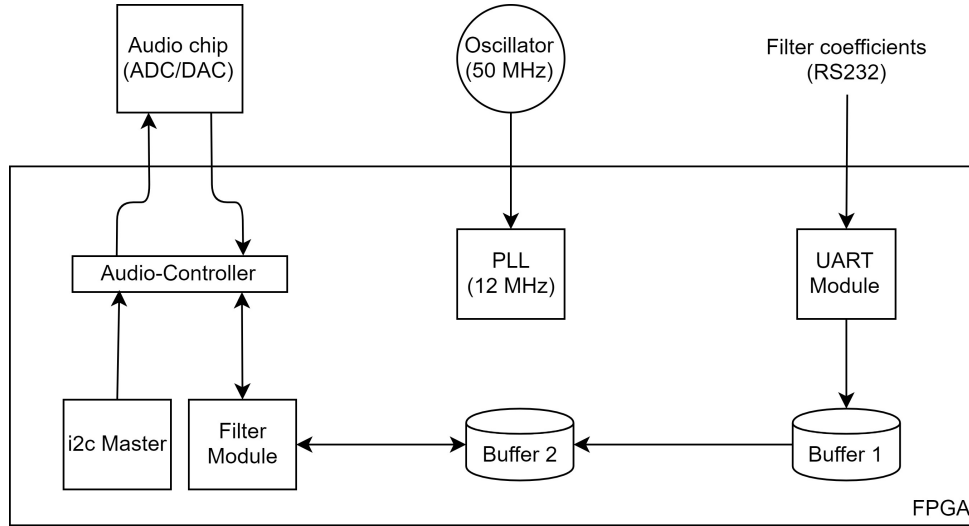


Fig. 16: Structure of the FPGA-Implementation

The audio controller serves as the interface for the i2c master and the filter module. The audio controller reads the individual bits that the audio chip transmits, buffers them and combines them to a 24 bit value. After the value has been read completely, it is transferred to the filter module and the filter calculation is started. The filter module uses floating point modules created using the IP Core Wizard to perform its calculations. After a few clock cycles, the calculation of the filter module is completed and the 24 bit output value is returned to the audio controller. There it is buffered and sent back to the audio chip at the next opportunity. This transfer happens again serially. The Communication between the audio chip and the FPGA is shown in Figure 17.

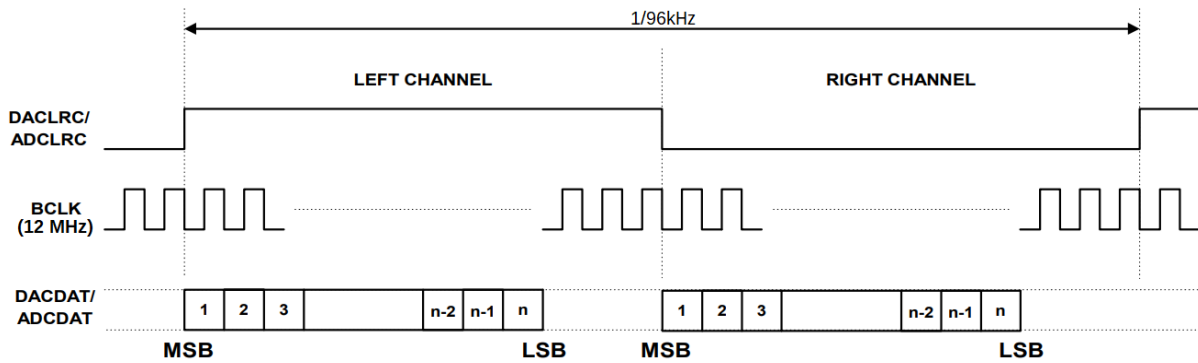


Fig. 17: Communication between the Audio-Chip and the FPGA [15]

On the rising edge of the 96 kHz signal, the individual 24 bits of the left input channel are sent to the FPGA at a speed of 12 MHz. In addition, the output value belonging to the previous input is also transmitted sequentially via a separate line at this time. This results in a delay between the input and output of one 96 kHz clock period = 10.42 μ s. The sample and hold time of the ADC and DAC is not taken into account here. The right signal is transmitted on the falling edge of the 96 kHz clock in the same way.

The FPGA receives the filter coefficients from the server via a serial interface (RS232). A

UART protocol is used for that. The received bytes are stored in buffer 1 and copied to buffer 2 when the transfer is completed and the filter module does not currently need to access the coefficients stored in buffer 2. DRAM blocks of the FPGA were used to build the buffers in order to make the implementation more energy-efficient. The buffers were created using the IP Core Wizard.

6.4 FPGA/Server-Communication

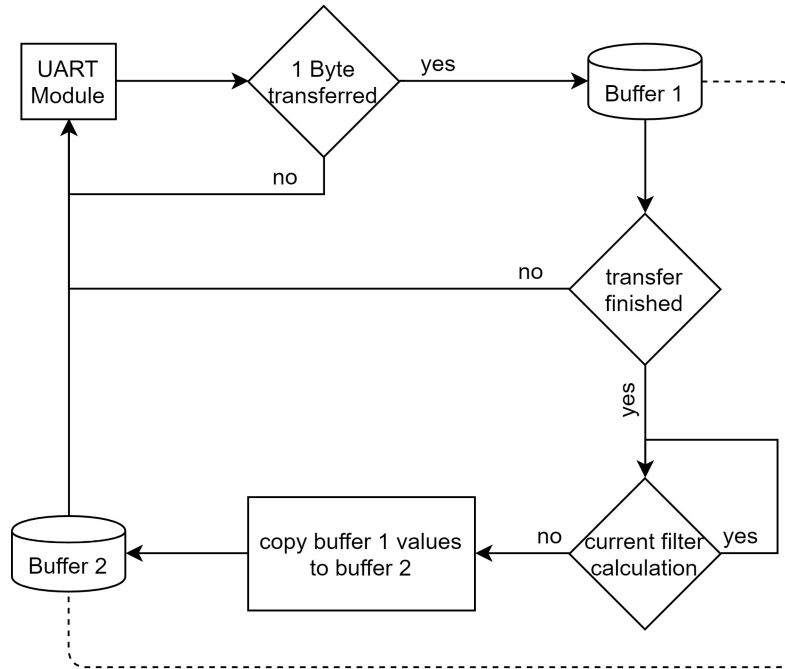


Fig. 18: FPGA-coefficients: reading and storing

As with the communication between the MCU and the server, the UART protocol with a baud rate of 9600 is used again. The communication is managed by the UART module. If a message, i.e. one byte, has been received completely, it is written to buffer 1. The next byte is written to the next address of buffer 1. If all 8 coefficients, i.e. all 64 bytes, have been transferred, it is checked whether the filter module is currently calculating. If this is not the case, the coefficients are copied in sequence from buffer 1 to buffer 2.

If the filter module is currently calculating or is going to start a calculation in the next 2 clock cycles, the system waits for the calculation to be completed. Only then the coefficients are transferred from buffer 1 to buffer 2. Otherwise it could occur that the filter uses partly new and partly old coefficients for its calculation. This in turn would lead to invalid output values and filter states, which are reflected in the output signal as minimal but nonetheless existing distortions.

If the copying of the contents of buffer 1 into buffer 2 has been permitted, data can be received in parallel via UART. This is possible because copying from buffer 1 to buffer 2 is much faster than writing to buffer 1. Furthermore, it is no problem if a calculation of the filter module is started during the copying process from buffer 1 to buffer 2.

It cannot happen that a new UART transfer takes place without the values from buffer 1 being copied into buffer 2, since the transfer between the MCU and the server takes a lot longer than

the calculation of the filter module.

6.5 FPGA-Filter Module

The filter module contains a state machine that carries out atomic floating point arithmetic operations. For this purpose, floating point calculation modules were created using the IP Core Wizard. An addition module, a multiplication module, an integer to floating point converter for converting the 24 bit input signal into a 64 bit floating point value and a floating point to integer converter for converting the calculated 64 bit floating point output value into a 24 bit integer value.

The modules are controlled so that they perform the entire calculation as quickly and energy-efficiently as possible. The multiplication module uses the filter coefficients stored in buffer 2 and the current filter states for the calculation. The addition module adds the results of the multiplication module to calculate the new filter states and the output.

6.6 DSP-Implementation

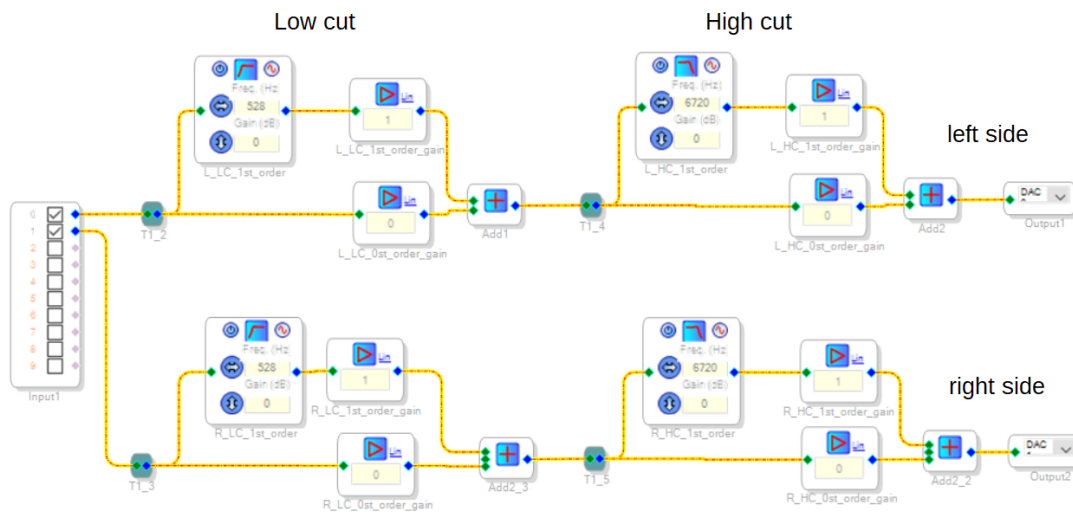


Fig. 19: DSP-Filterdesign

Unlike *Quartus II*, which was used to design for FPGA implementation, *Sigma Studio* offers all the objects required for this task. These only have to be selected from a list and linked. The correct parameters must also be specified. E.g., an ADC and DAC resolution of 96 kHz and 24 bit was chosen again.

As an alternative to the graphical user interface of Sigma Studio shown in Figure 19, the design can also be implemented using Sigma Studio Scripts. Sigma Studio Scripts can be written in C# and Visual Basic. Because only components of Sigma Studio were connected in this implementation, the implementation is not dealt with in more detail here. For further information, please refer to various documentation from Sigma Studio and Analog Devices DSP chips.

A Sigma Studio Script written in C# is used for the automatic reconfiguration of the DSP. The server receives, as described before the cut off frequencies from the MCU and passes them to Sigma Studio. Synchronisation plays an important role here. The Sigma Studio Script uses those cut off frequencies to reconfigure the project and uploads the new design to the DSP. This is done without a break of the output signal.

7 Result

7.1 Noise Reduction - Measurement Setup

First of all a few words about the following graphs. The left side of the figures are measurements of the DSP system, the right side are measurements of the FPGA system. The upper side are measurements with the filter deactivated, the lower side measurements with the filter activated. The image description shows the automatically calculated and applied filter cut-off frequencies and the signal noise ratio. The blue signal corresponds to the input signal, the red one to the output signal. A standard oscilloscope and a function generator were used for the measurements.

The reason why the input noise in case of the FPGA implementation looks stronger than the input noise in case of the DSP implementation is that the sampling frequency of the oscilloscope was set to 1 MHz when measuring the DSP board. The sampling frequency in the case of the FPGA board was 100 MHz. Accordingly, higher noise frequencies can be measured too.

According to section *Background* both systems cut frequencies above 24 kHz. Accordingly the difference in sampling frequency is only notable by a stronger looking Input noise. But the behaviour of the systems and the measurements of the output signal are not influenced by the difference in sampling frequency.

7.2 Noise Reduction - 100 Hz

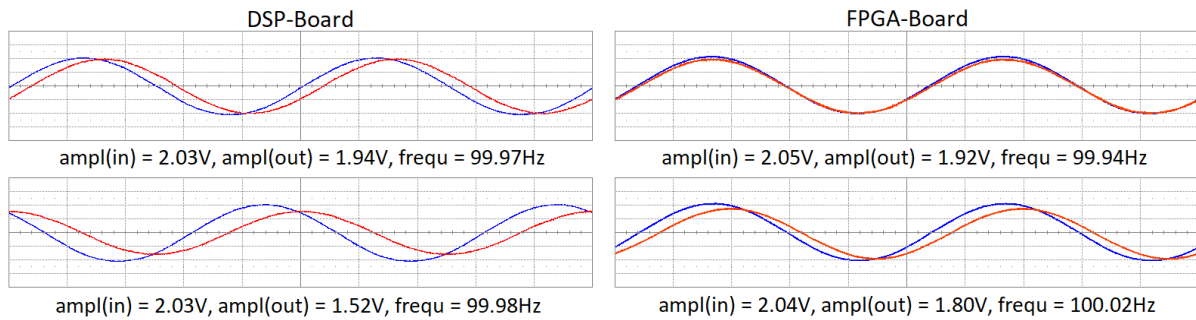


Fig. 20: noise component: 0%
cut-off frequencies: 0 and 288 Hz

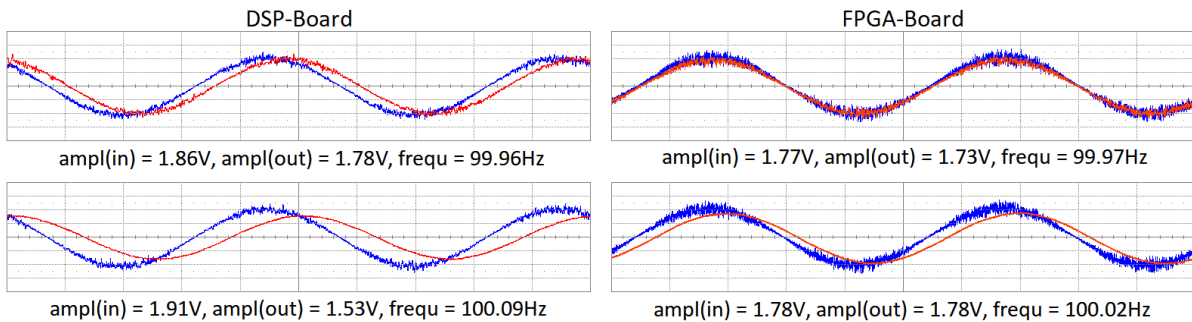


Fig. 21: noise component: 60%
cut-off frequencies: 0 and 288 Hz

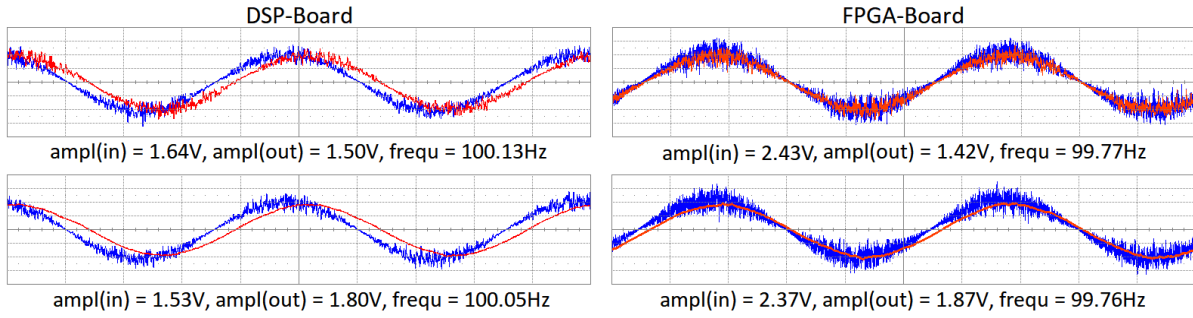


Fig. 22: noise component: 120%
cut-off frequencies: 0 and 672 Hz

At low frequencies, the FPGA board and the DSP board behave practically the same. Only the amount and phase differ due to the frequency responses of the systems described in section *Overview*. The upper cut-off frequency is optimally calculated up to a noise component of 100%. After that, it is 672Hz instead of 288Hz, which only slightly worsens the result. A slight distortion of the filtered output signal is only noticeable at very strong noise.

7.3 Noise Reduction - 1 kHz

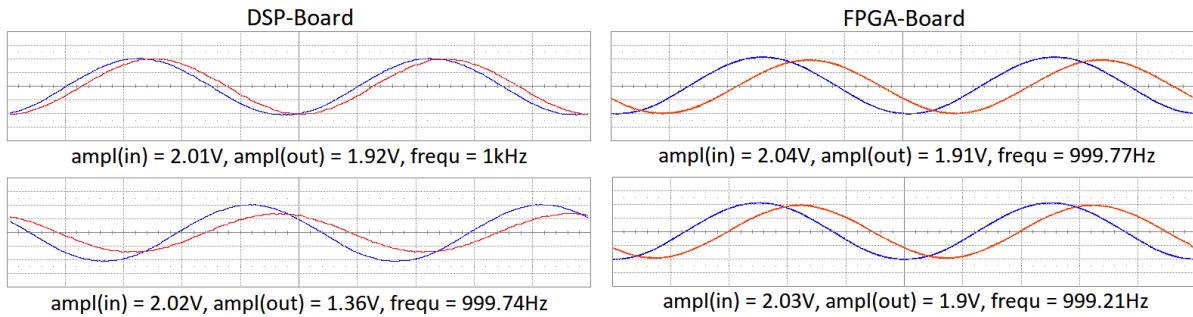


Fig. 23: noise component: 0%
cut-off frequencies: 576 Hz and 2304 Hz

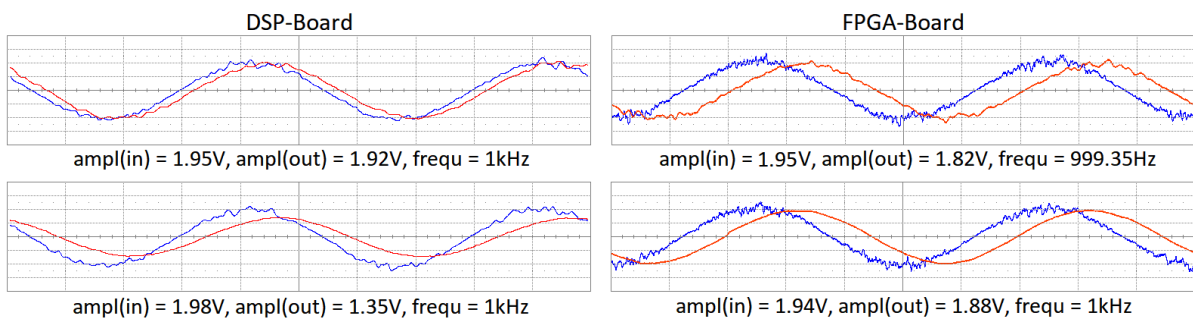


Fig. 24: noise component: 60%
cut-off frequencies: 576 Hz and 2304 Hz

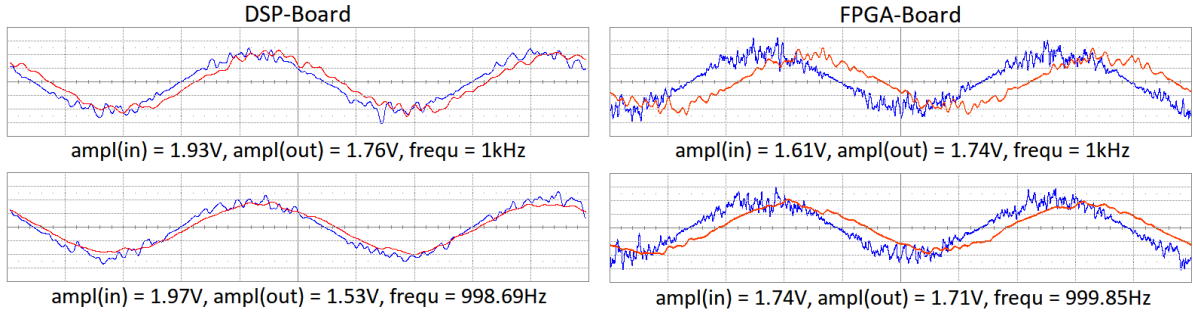


Fig. 25: noise component: 120%
cut-off frequencies: 576 Hz and 6720 Hz

The two systems behave approximately the same even at medium frequencies. At this frequency of the input signal, the higher cut off frequency is optimally calculated up to a noise component of 90%. With very strong noise, slight distortions can be detected here, despite filtering. Nevertheless, the filtered signal is still significantly less noisy than in the unfiltered case.

7.4 Noise Reduction - 10 kHz

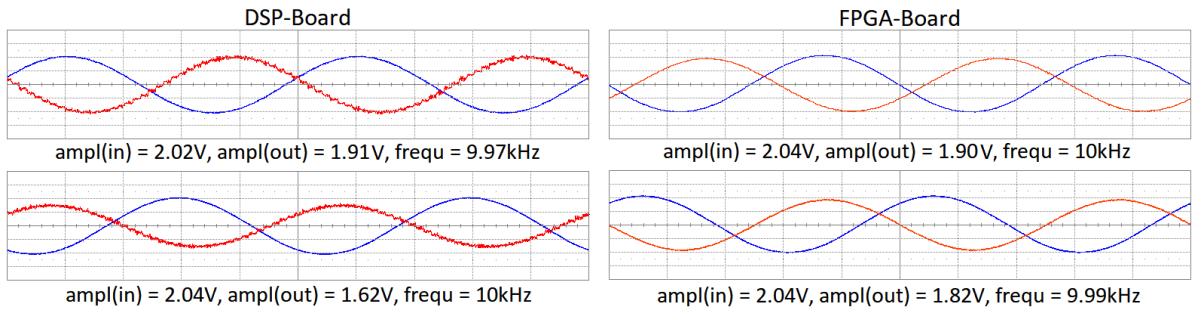


Fig. 26: noise component: 0%
cut-off frequencies: 6240 Hz and 22.37 kHz

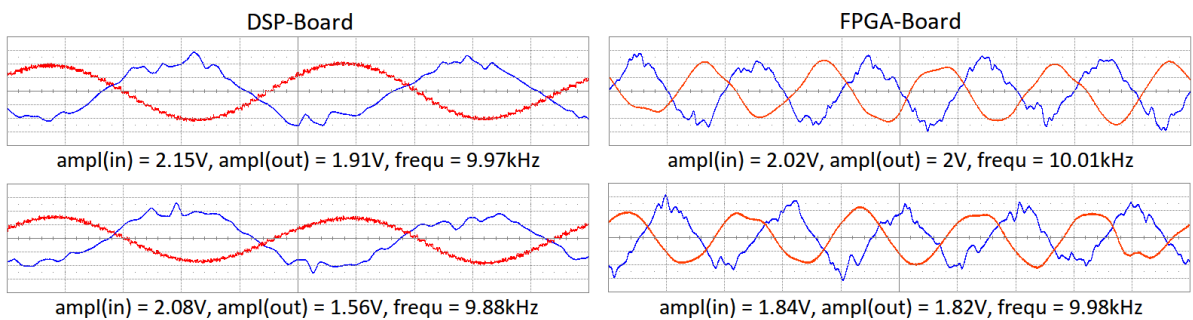


Fig. 27: noise component: 120%
cut-off frequencies: 6240 Hz and 22.37 kHz

At input frequencies that are relatively high for audio conditions, the upper cut off frequency is set so that it has practically no effect. But what can be noted in the measurements shown in this subsection is that in the case of the DSP board, noise is almost completely filtered. This is

not the case with the FPGA board. This behavior can be explained by the bode diagrams in section *Background*. As already mentioned there, in the case of the DSP board, frequencies from 22 kHz are cut off cleanly. In the case of the FPGA board, frequencies are cut off cleanly from 27 kHz. Clean refers to a high slope rate. Accordingly, higher frequencies can pass at the FPGA system.

Furthermore, in the case of the DSP board, the quantization error can be recognized. This is due to the limited sampling frequency of 96 kHz. Also noise generated by the DAC is visible. The WM8731, which is built into the FPGA board, has an additional output low pass. This smoothes the output signal. The DSP board responds better to high-frequency input noise, but does not have an output low-pass filter. However, this can simply be added afterwards.

7.5 Noise Reduction - other Waveforms

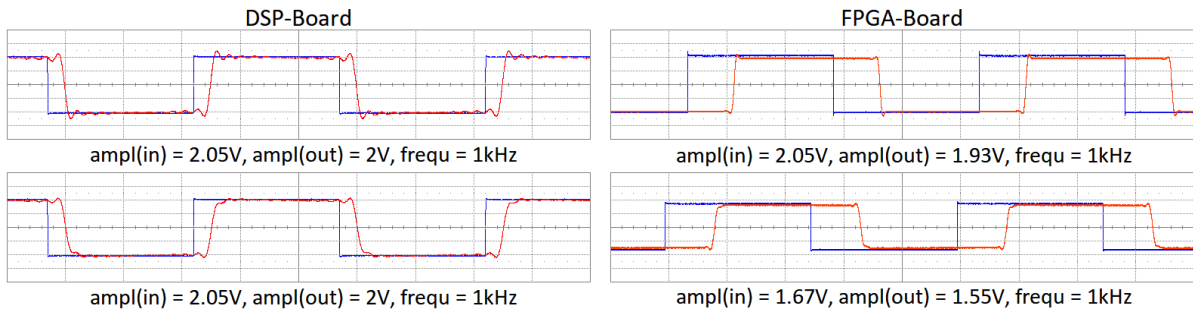


Fig. 28: noise component: 0%
cut-off frequencies: 0 and 20.06 kHz

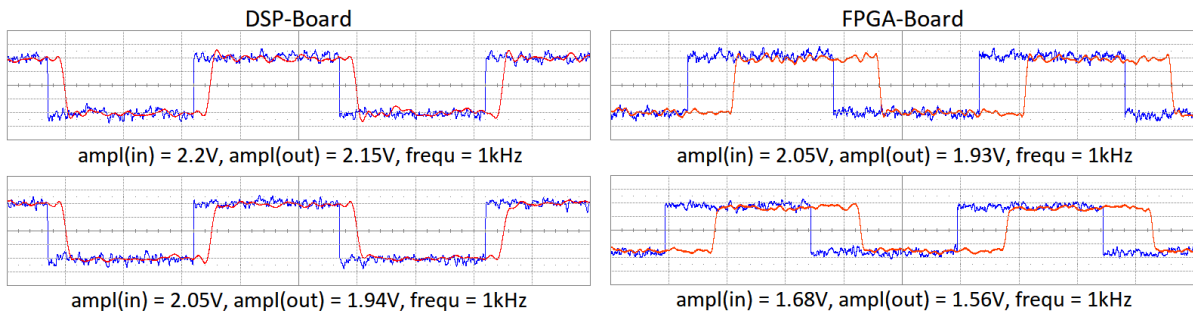


Fig. 29: noise component: 60%
cut-off frequencies: 0 and 20.06 kHz

As you can see in this subsection, filtering of complex signals brings only minor improvements. Those signals, e.g. square waves or saw waves have relatively strong harmonics in addition to the fundamental oscillation. The algorithm is designed in a way that the harmonics are not filtered out. Accordingly, the upper cut-off frequency is set relatively high.

With a slight adaptation of the algorithm, as described in section *Methodology*, the sensitivity can be reduced. This would result in a lower upper cut-off frequency in the case of the more complex signals. This would reduce the noise, but also important harmonics would be attenuated.

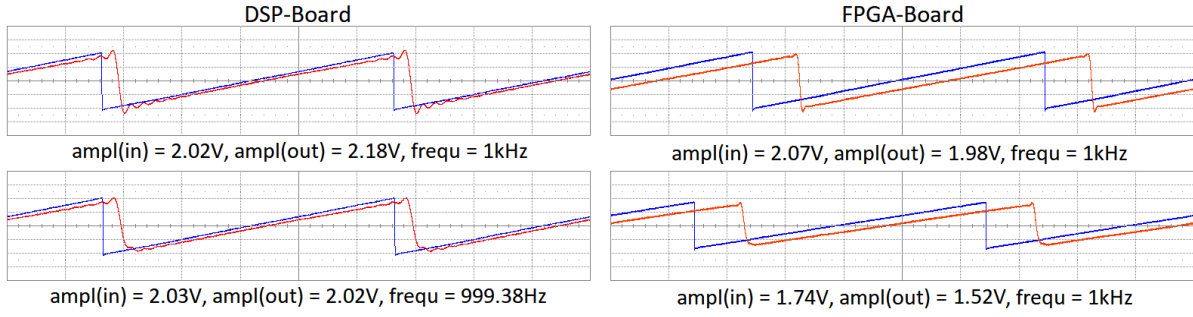


Fig. 30: noise component: 0%
cut-off frequencies: 0 and 17.86 kHz

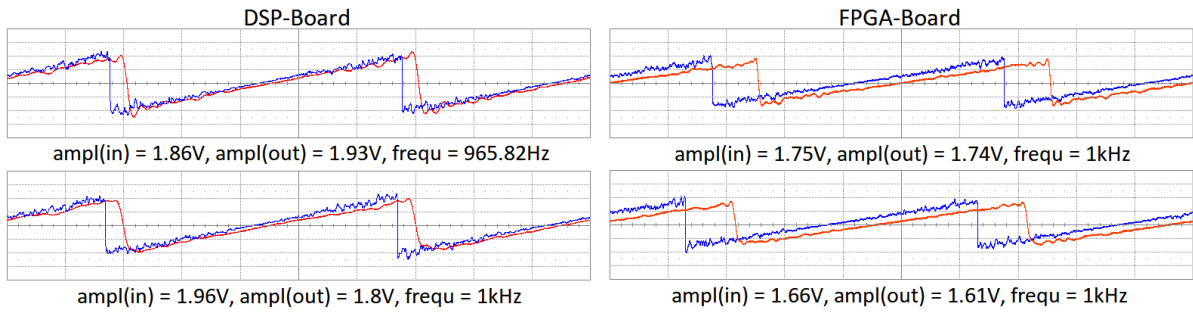


Fig. 31: noise component: 60%
cut-off frequencies: 0 and 17.86 kHz

In order not to destroy the signal shape, it is also advisable to disable the high-pass filtering. For this reason, the lower cut-off frequency was set to 0 at the measurements listed in this subsection.

7.6 Reconfiguration Timing and Latency

The reconfiguration time of the two implementations is currently around 200 ms. This means that it usually takes about 200 ms for the filter instrument to respond to a frequency change in the input signal. This reconfiguration time is sufficient for the requirements, even if it could be slightly audible. If you still want to optimize, it is recommended to increase the transmission speed of the serial communication first. However, there is no guarantee that the functionality will continue working.

The transmission speed for each of the three serial transmissions is currently 9600 bits/s. With a start bit, 8 data bits, a parity bit and a stop bit. I.e. 11 bits transmitted per UART transmission. This results in a combined transmission time of:

Device	data [bits]	total data [bits]	duration [ms]
Server - MCU	8	11	1.15
MCU - Server	96	132	13.75
Server - FPGA/DSP	512	704	73.33
		sum:	88.23

In addition to the 88.23 ms that the entire transmission takes, the calculation and buffering of the MCU and the calculation of the server play a central role. Unlike the transmission, these

processes cannot be optimized so easily.

The latency is defined by the sampling and hold time of the ADC and DAC and the response time of the systems. The response time of the FPGA-Implementation is exactly one 96 kHz period. This leads to a latency of about 20 μ s. In the case of the DSP-Implementation similar behavior can be observed. This short latency is not recognisable by human hearing.

7.7 Effort and Expandability

The effort to create the FPGA implementation was much higher than the effort to create the DSP implementation. Even if the number of lines of code hardly says anything about the actual effort of an implementation, the difference is obvious in this case.

The following table compares the number of lines of written code for both implementations. This does not include inserted VHDL-IP cores, methods that have not been developed by us and also no VHDL test benches. In addition, the effort of the MCU implementation is not included in this calculation, since the FPGA and the DSP implementation use the same MCU system.

Language	FPGA	DSP
VHDL	1540	0
Python	70	0
Matlab	104	0
C	0	143
Bash	18	15
sum:	1732	158

It should also be mentioned that the effort of the FPGA implementation due to the testing and verification of the very large code base was accordingly complex. A better indicator for the effort of a design are the hours of developing. The FPGA implementation took approximately 160 hours of understanding, designing and testing. On the other hand the DSP implementation was finished in about 20 hours. Furthermore the designing of the DSP implementation needs almost no foreknowledge. That cannot be said about designing FPGAs using VHDL.

A significant advantage of the FPGA implementation is the expandability. The Wondom DSP board is a closed system. It is not freely programmable, but can only be configured directly using Sigma Studio. It is limited to the functionalities that can be defined in Sigma Studio. In addition, Sigma Studio is not available for Unix systems which is the main reason why we're using an Windows PC as server in this project.

However there is a possibility to export a C code generated by Sigma Studio which allows the reconfiguration of parameters of objects primarily build in Sigma Studio. This allows the reconfiguration of such a DSP device without a Windows PC. e.g. a MCU. The FPGA board, however, can be expanded as desired.

7.8 Real Use Case

The application area of the implementation is wide spreaded. It can be used wherever a relatively simple and band-limited signal, for example the output signal of a sensor or a musical instrument, is to be freed from noise. The main area of application in the development were audio-technical signals, primarily the output signals of musical instruments. It should be noted that signals that are only audible to humans, i.e. those that are in a frequency range between 20 Hz and 20 kHz,

can be correctly handled by the system.

Imagine a analogue synthesizer. Because of a noise source in the area close to the synthesizer, the output signal of it is overlaid with noise. Between the synthesizer and the mixer is our system located. The MCU analyzes the signal coming from the synthesizer and calculates a corresponding band limitation. This band limitation is used to configure the filter device. The filter device filters the output signal from the synthesizer and sends a less noisy signal to the mixer with a latency caused by the filter of only 20 μ s. This latency is not recognisable by human hearing.

8 Conclusion

In this work two different implementations of a dynamic digital filter for automatic noise reduction were presented. The system was developed with an emphasis on audio signals, but can also be used for filtering other signals in the corresponding frequency range.

A DSP board was used for one of the implementations and a FPGA board for the other. The main difference here, as described in the section *Result*, is the effort and extensibility of the implementations. The DSP implementation was completed in a fraction of the time required for the FPGA implementation. But the FPGA implementation is a lot more flexible and can also be expanded in areas that have nothing to do with the processing of audio signals.

In the section *Result* further differences of the implementations were emphasized. Both systems work equally well at most input frequencies. Only the amplitude and phase of the filtered system is slightly different due to the frequency responses of the systems described in the section *Background*. In addition, very high-frequency noise is better filtered by the DSP board than by the FPGA board.

The code designed for this project is visible on git lab [17].

REFERENCES

- [1] Richard S Burwen, "Variable bandpass dynamic noise filter", US-Patent, 03 November 1970
- [2] Richard S Burwen, "Dynamic noise filter having means for varying cutoff point", US-Patent, 03 November 1970
- [3] Wesley W. Murphy III, James Hettiger, "Dynamic noise filter for an audio signal in a television", US-Patent, 18 October 1982
- [4] Nobuaki, NaoiKatunobu, TakedaIkuo Yuki, "Dynamic noise reduction circuit for image luminance signal", JP-Patent, 07 April 1988
- [5] Shuvra S. Bhattacharyya, "Methods for design and implementation of dynamic signal processing systems", 2011 International Conference on Embedded Computer Systems: Architectures, Modeling and Simulation, 18-21 July 2011
- [6] Abdolkarim Hashemi, Masoomah Rahimpour, Mahmoud Reza Merati, "Dynamic Gaussian filter for muscle noise reduction in ECG signal", 2015 23rd Iranian Conference on Electrical Engineering, 10-14 May 2015
- [7] Hina Magsi, Ali Hassan Sodhro, Faheem Akhtar Chachar, Saeed A. Khan Abro, "Analysis of signal noise reduction by using filters", 2018 International Conference on Computing, Mathematics and Engineering Technologies (iCoMET), 3-4 March 2018
- [8] Tushar Kanti Roy, Monir Morshed, "Active noise control using filtered-xLMS and feedback ANC filter algorithms", 2013 2nd International Conference on Advances in Electrical Engineering (ICAEE), 19-21 Dec. 2013
- [9] Adam S. Charles, Aurele Balavoine, Christopher J. Rozell, "Dynamic Filtering of Time-Varying Sparse Signals via ℓ_1 Minimization", IEEE Transactions on Signal Processing, 30 June 2016
- [10] Chun-Cheng Liu, Mu-Chen Huang, "28.1 A 0.46mW 5MHz-BW 79.7dB-SNDR noise-shaping SAR ADC with dynamic-amplifier-based FIR-IIR filter", 2017 IEEE International Solid-State Circuits Conference (ISSCC), 5-9 Feb. 2017
- [11] K. Singh, "Noise reduction in analog to digital converters", Proceedings of the International Conference on Electromagnetic Interference and Compatibility, 8 Dec. 1999
- [12] DE2-70 User Manual
- [13] www.boomaudio.de/wondom-dsp-board, accessed on: 02/03/2020, 11:30
- [14] www.pjrc.com/store/teensy36.html, accessed on: 02/03/2020, 12:27
- [15] Rauschen von Rudolf Müller, 1990, Springer Verlag
- [16] Electronic Noise and Interfering Signals: Principles and Applications from Gabriel Vasilescu, 2005, Springer Verlag
- [17] <https://gitlab.com/tzuzuzj/dynamic-configuration-platforms-for-digital-noise-filtering-of-signals>, accessed on 09/04/2020, 12:38